

Lab class 2 Exercise Sheet

Model answers (questions 3 and 4)

Question 3:

A network has a mean degree of 4 and a mean excess degree of 6, then half the nodes are removed at random. What is your expectation for the mean degree and mean excess degree after the nodes have been removed? Before using any code, try to predict the answer. Once you have your prediction, use code above to check whether your prediction is correct. To be clear, you will not get a network with a mean degree of 4 and a mean excess degree of 6, but using those 3 networks, you will learn that the impact is identical between all 3 types of networks. Now, to get better results, make the networks sufficiently large (the larger, the better).

For the hypothetical network considered (which should be pretty much a BA network as configured in question 2), the mean degree will be 2, and the mean excess degree will be 3. Removing a proportion of the nodes at random reduces the expectation value for the mean degree and the mean excess degree of a network by the same proportion.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

# Function to calculate mean degree
def mean_degree(G):
    degrees = [deg for _, deg in G.degree()]
    return np.mean(degrees)

# Function to calculate mean excess degree
def mean_excess_degree(G):
    degrees = np.array([deg for _, deg in G.degree()])
    if np.sum(degrees) == 0:
        return 0 # Avoid division by zero
    return np.sum(degrees * (degrees - 1)) / np.sum(degrees)
```

```

# Function to remove half of the nodes at random
def remove_half_nodes(G):
    nodes_to_remove = np.random.choice(G.nodes(), size=len(G) // 2, replace=False)
    G.remove_nodes_from(nodes_to_remove)
    return G

num_experiments = 100
mean_degrees_before = []
mean_excess_degrees_before = []
mean_degrees_after = []
mean_excess_degrees_after = []

for _ in range(num_experiments):
    # Generate a large Barabási-Albert (BA) network with mean degree 4
    N_large = 2000
    m_ba = 2 # BA model parameter
    ba_graph_large = nx.barabasi_albert_graph(n=N_large, m=m_ba)

    # Calculate mean degree and mean excess degree before removing nodes
    ba_mean_k_before = mean_degree(ba_graph_large)
    ba_mean_excess_k_before = mean_excess_degree(ba_graph_large)

    # Remove half of the nodes at random
    ba_graph_large = remove_half_nodes(ba_graph_large)

    # Calculate mean degree and mean excess degree after removing nodes
    ba_mean_k_after = mean_degree(ba_graph_large)
    ba_mean_excess_k_after = mean_excess_degree(ba_graph_large)

    # Store results
    mean_degrees_before.append(ba_mean_k_before)
    mean_excess_degrees_before.append(ba_mean_excess_k_before)
    mean_degrees_after.append(ba_mean_k_after)
    mean_excess_degrees_after.append(ba_mean_excess_k_after)

# Calculate mean values
mean_degree_before = np.mean(mean_degrees_before)
mean_excess_degree_before = np.mean(mean_excess_degrees_before)
mean_degree_after = np.mean(mean_degrees_after)
mean_excess_degree_after = np.mean(mean_excess_degrees_after)

# Calculate ratios

```

```

ratio_mean_degree = mean_degree_after / mean_degree_before
ratio_mean_excess_degree = mean_excess_degree_after / mean_excess_degree_before

# Print results
print(f"Before removal: Mean Degree = {mean_degree_before:.2f}, \
      Mean Excess Degree = {mean_excess_degree_before:.2f}")
print(f"After removal: Mean Degree = {mean_degree_after:.2f}, \
      Mean Excess Degree = {mean_excess_degree_after:.2f}")
print(f"Ratio (After/Before) - Mean Degree: {ratio_mean_degree:.2f}, \
      Mean Excess Degree: {ratio_mean_excess_degree:.2f}")

```

```

Before removal: Mean Degree = 4.00,    Mean Excess Degree = 10.88
After removal: Mean Degree = 1.97,    Mean Excess Degree = 5.12
Ratio (After/Before) - Mean Degree: 0.49,    Mean Excess Degree: 0.47

```

In both cases, the ratio should be $1/2$ (or very close to).

Question 4: Programming exercise

In the next lecture, we will introduce a property called “degree assortativity”, which, put simply, is a measure of the tendency of nodes to connect to other nodes with similar degree. Practically, degree assortativity is calculated using the assortativity coefficient r , which ranges from -1 (perfect disassortativity) to +1 (perfect assortativity), with 0 indicating no correlation. To calculate the degree assortativity of a network G , use `nx.degree_assortativity_coefficient(G)`. In this exercise, I am asking you to write code to generate your own low- and high-assortativity networks (say with 100 nodes, i.e., I am not interested in a small hand-designed graph). Feel free to come up with your own approach but one idea might be to start from some kind of random network (and here you might want to pick a suitable form of random graph depending on whether you want high or low assortativity) and then do some *rewiring* to tune the assortativity up or down. (*hint: the principle of a rewiring algorithm is that you will pick two pairs of nodes, one that is connected by an edge and one that isn't, and then, delete the existing edge and add a new edge if it will either increase or decrease assortativity*)

For the low assortativity network, we will start from a kind of random network that is already naturally disassortative. That is the ‘scale-free graph’. It is naturally disassortative because hubs have a very high degree and are connected to many nodes of low degree. Then, we pick 4 nodes (u,v,x,y) , 2 of which (u,v) are connected by an edge for sure. Then, if the difference in degree between nodes (u,v) which are connected by the edge is less than the difference in degree between the nodes (x,y) who are not connected by an edge, then we delete the edge

between u and v , and we create an edge between x and y . This increases disassortativity by decreasing the correlation between node degrees.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

def generate_low_assortativity_network(n=100, m=3, rewiring_attempts=1000):
    """
    Generates a network with low (disassortative) assortativity.

    - Starts with a scale-free Barabási-Albert graph.
    - Rewires edges to decrease degree similarity among connected nodes.
    """
    G = nx.barabasi_albert_graph(n, m) # Initial scale-free network

    for _ in range(rewiring_attempts):
        edges = list(G.edges())
        np.random.shuffle(edges)
        for u, v in edges:
            # Select two random nodes
            x, y = np.random.choice(list(G.nodes()), 2, replace=False)

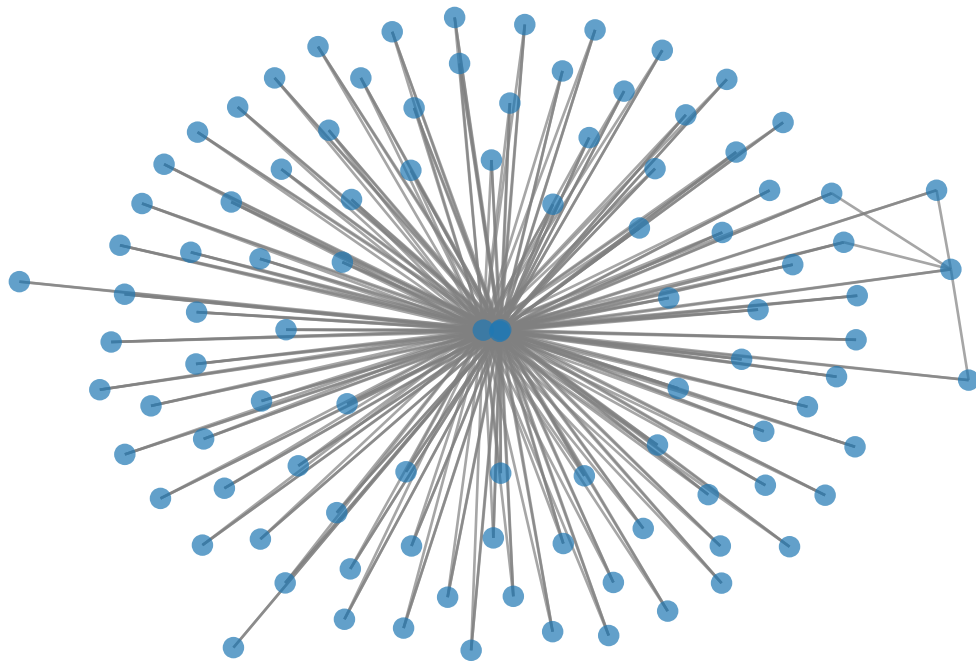
            if G.has_edge(u, v) and not G.has_edge(x, y):
                # Swap edges only if it decreases degree similarity
                if abs(G.degree[u] - G.degree[v]) < abs(G.degree[x] - G.degree[y]):
                    G.remove_edge(u, v)
                    G.add_edge(x, y)

    # Calculate assortativity
    assortativity = nx.degree_assortativity_coefficient(G)
    return G, assortativity

# Generate the low-assortativity network
G_disassort, assort_value = generate_low_assortativity_network()

# Draw the network
plt.figure(figsize=(6, 4))
nx.draw_spring(G_disassort, node_size=50, edge_color='gray', alpha=0.7, with_labels=False)
plt.show()

# Display assortativity value
print(f"Low Assortativity (Disassortative) Network: {assort_value:.3f}")
```



Low Assortativity (Disassortative) Network: -0.967

For the high-assortativity network, we start from the Erdos-Renyi random network as, unlike the scale-free network, it will not be naturally disassortative. Then, we use the same approach as above but now we only remove the edge if the difference in degree between unconnected nodes x and y is less than the difference between connected edges u and v . In other words, we try to make the degrees closer, i.e., higher correlation.

```
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt

def generate_high_assortativity_network(n=100, p=0.2, rewiring_attempts=1000):
    """
    Generates a network with high assortativity, where nodes with similar degrees
    are more likely to be connected.
```

```

- Starts with an Erdős-Rényi random graph.
- Rewires edges to increase degree similarity among connected nodes.
"""
G = nx.erdos_renyi_graph(n, p) # Start with a random graph

for _ in range(rewiring_attempts):
    edges = list(G.edges())
    np.random.shuffle(edges)
    for u, v in edges:
        # Select two random edges
        x, y = np.random.choice(list(G.nodes()), 2, replace=False)

        if G.has_edge(u, v) and not G.has_edge(x, y):
            # Swap edges only if it increases degree similarity
            if abs(G.degree[u] - G.degree[v]) > abs(G.degree[x] - G.degree[y]):
                G.remove_edge(u, v)
                G.add_edge(x, y)

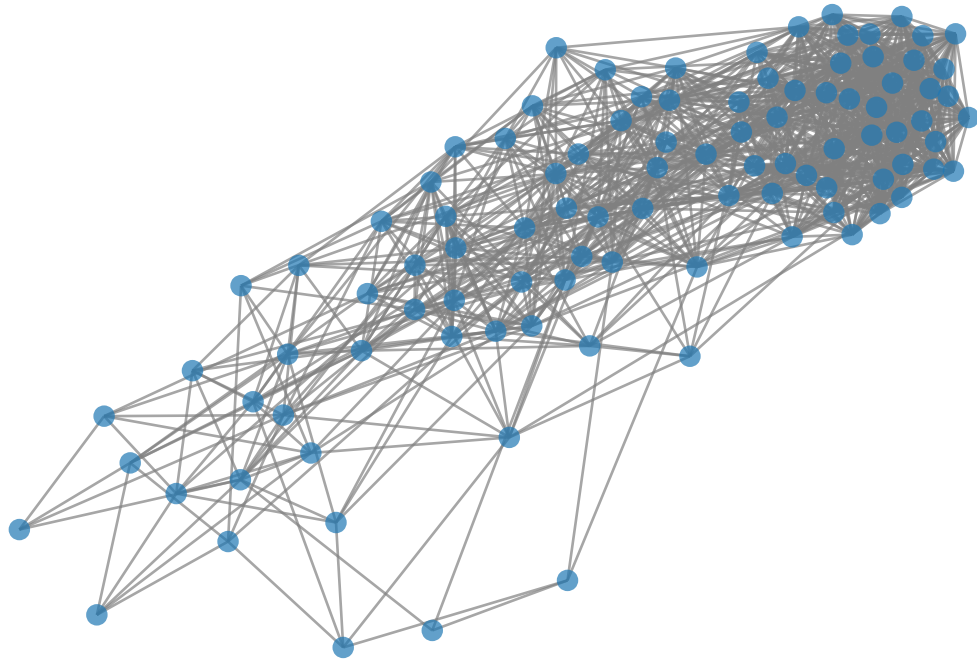
    # Calculate assortativity
    assortativity = nx.degree_assortativity_coefficient(G)
    return G, assortativity

# Generate the high-assortativity network
G_assort, assort_value = generate_high_assortativity_network()

# Draw the network
plt.figure(figsize=(6, 4))
nx.draw_spring(G_assort, node_size=50, edge_color='gray', alpha=0.7, with_labels=False)
plt.show()

# Display assortativity value
print(f"High Assortativity Network: {assort_value:.3f}")

```



High Assortativity Network: 0.708